ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Semestrálna práca z predmetu

Diskrétna simulácia

Vypracoval: Bc. Richard Vymazal

Študijná skupina: 5ZIA12

cvičiaci: Ing. Andrea Galadíková, PhD.

Termín cvičenia: piatok bloky 7 – 8

v Žiline dňa 13.3.2026

Obsah

Úvod 3
Testovanie generátora spojitého a diskrétneho empirického rozdelenia 3
Opis modelu 4
Cestná sieť 4
Pravdepodobnostné rozdelenia rýchlostí 4
Diskrétne rovnomerné rozdelenie 5
Spojité rovnomerné rozdelenie 5
Spojité empirické rozdelenie 5
Diskrétne empirické rozdelenie 5
Dopravné spomalenie v bode K 5
Simulácia obslúženia jednotlivých obcí 6
Princíp simulácie 6
Výsledky simulácie 6
Simulácia optimálneho odchodu 7
Princíp simulácie 7
Hľadanie optimálneho času odchodu 7
Výsledky simulácie 8
Architektrúra systému 8
Simulačné jadro 8
Implementované simulácie 9
Diagram tried 9
Grafické používateľské rozhranie 10
Záver 11
Využitie AI 12

Úvod

Cieľom tejto semestrálnej práce je vytvoriť simulačný model rozvozu potravín z veľkoskladu v meste Žilina do troch predajní nachádzajúcich sa v obciach Strečno, Rajecké Teplice a Divinka. Model je implementovaný pomocou metódy Monte Carlo a jeho úlohou je analyzovať správanie systému pri náhodných rýchlostiach jazdy na jednotlivých úsekoch ciest.

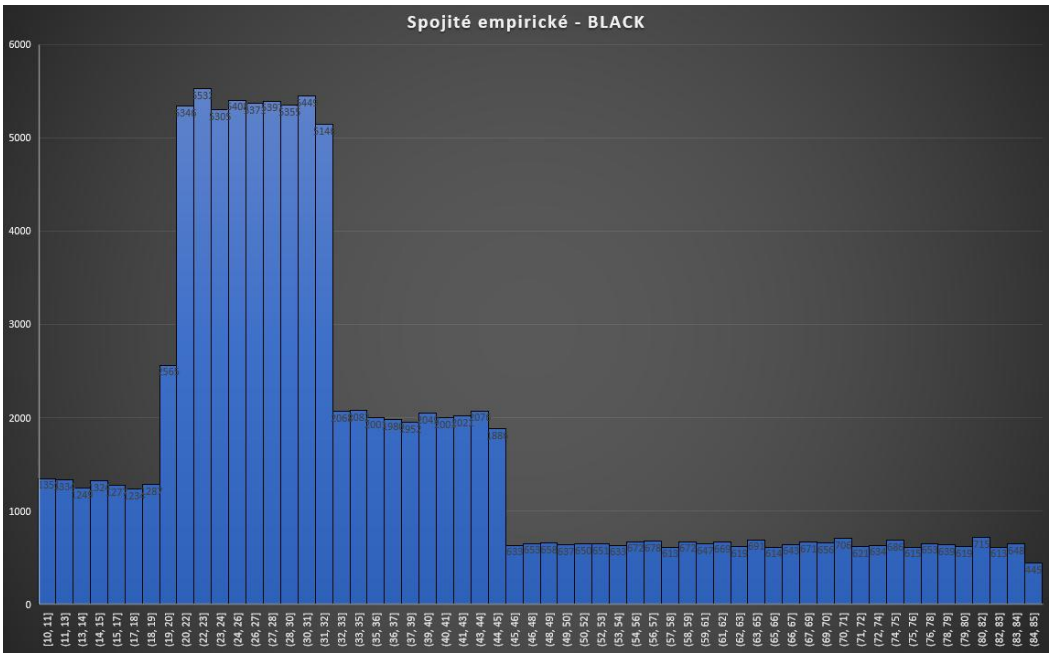
Simulácia rieši dve hlavné úlohy:

1. Určiť optimálne poradie obsluhy jednotlivých obcí tak, aby kuriér priemerný čas rozvozu minimalizoval.
2. Určiť optimálny čas odchodu kuriéra zo skladu tak, aby s čo najpresnejšou pravdepodobnosťou stihol dokončiť rozvoz pred stanoveným časovým limitom.

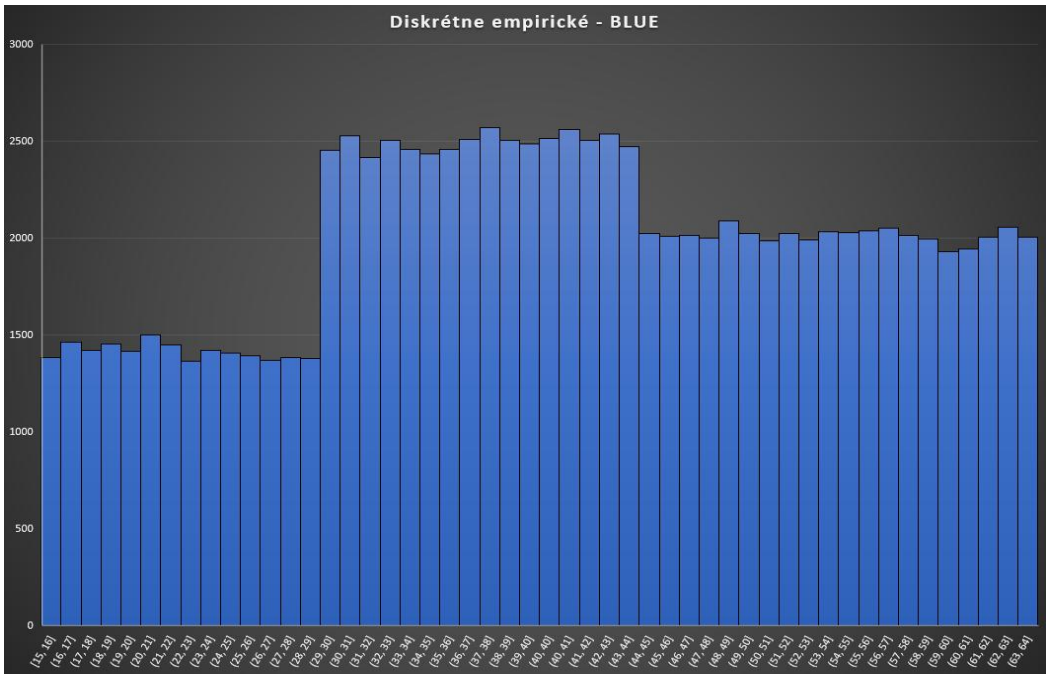
Pri riešení sa využíva simulačné jadro implementované ako všeobecný *Monte Carlo* framework, ktorý vykonáva veľký počet replikácií experimentu. Každá replikácia predstavuje jednu možnú realizáciu systému s náhodne generovanými rýchlosťami vozidla na jednotlivých cestných úsekoch.

Testovanie generátora spojitého a diskrétneho empirického rozdelenia

Cieľom testovania bolo overiť, či implementácie distribúcií *Black* (spojité empirické rozdelenie) a *Blue* (diskrétne empirické rozdelenie) generujú hodnoty v súlade so zadaním. Testovanie prebehlo generovaním veľkého počtu náhodných hodnôt, ich exportom do .csv súboru a následnou analýzou pomocou grafov v programe MS Excel.



1 Spojité empirické rozdelenie



2 Diskrétne empirické rozdelenie

Opis modelu

Simulovaný systém predstavuje kuriéra, ktorý každý deň rozváža potraviny zo skladu v Žiline do obcí:

- Divinka
- Strečno
- Rajecké Teplice

Po obslúžení všetkých obcí sa kuriér vracia späť do Žiliny.

Cestná sieť

Model cestnej siete je reprezentovaný ako graf:

- **Vrchol** (*Node*) reprezentuje mesto alebo križovatku
- **Hrana** (*RoadSegment*) reprezentuje cestný úsek medzi dvoma vrcholmi

Každý cestný úsek obsahuje:

- vzdialenosť v kilometroch
- pravdepodobnostné rozdelenie rýchlosti

Vrchol môže byť typu:

- **CITY** – mesto, ktoré musí kuriér obslúžiť
- **JUNCTION** – križovatka
- **SLOW NODE** – špeciálna križovatka K s dopravným spomalením

Pravdepodobnostné rozdelenia rýchlostí

Rýchlosť na jednotlivých úsekoch je generovaná pomocou pravdepodobnostných rozdelení.

Diskrétne rovnomerné rozdelenie

Úsek	červený
Priemerná rýchlosť	Umin = 55 km/h Umax = 75 km/h

Spojité rovnomerné rozdelenie

Úsek	zelený
Priemerná rýchlosť	<50 , 80) km/h

Spojité empirické rozdelenie

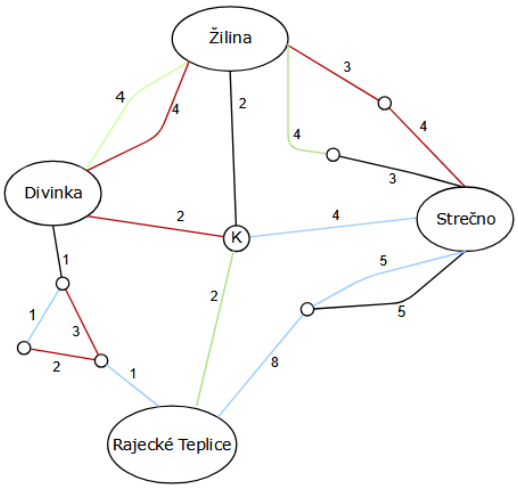
Úsek	čierny	pravdepodobnosť
Priemerná rýchlosť	<10, 20) km/h	0.1
	<20, 32) km/h	0.5
	<32, 45) km/h	0.2
	<45, 75) km/h	0.15
	<75, 85) km/h	0.05

Diskrétne empirické rozdelenie

Úsek	modrý	pravdepodobnosť
Priemerná rýchlosť	<15, 29) km/h	0.2
	<29, 45) km/h	0.4
	<45, 65) km/h	0.4

Dopravné spomalenie v bode K

V uzle *K* vznikajú dopravné kolóny po čase *6:30*. Ak kuriér opúšťa tento bod po tomto čase, rýchlosť sa znižuje o náhodnú hodnotu medzi 10% až 25%. Toto spomalenie je modelované spojitým rovnomerným rozdelením.



3 Graf cestnej siete

Simulácia obslúženia jednotlivých obcí

Cieľom simulácie je zistiť priemerný čas návratu kuriéra späť do Žiliny pre jednotlivé možné poradia obsluhy obcí. Keďže rýchlosť na jednotlivých cestných úsekoch je náhodná a riadi sa pravdepodobnostnými rozdeleniami, nie je možné presne určiť čas prejazdu analyticky.

Simulácia je vykonaná samostatne pre každé možné poradie obsluhy obcí. Keďže kuriér musí navštíviť tri obce (Divinka, Strečno a Rajecké Teplice) a každú obec navštíviť presne raz, existuje spolu šesť možných trás. Každá trasa začína aj končí v meste Žilina.

Princíp simulácie

Simulácia je implementovaná pomocou metódy Monte Carlo, pričom každá replikácia predstavuje jednu možnú realizáciu rozvozu. V každej replikácii sa náhodne generujú rýchlosti na jednotlivých cestných úsekoch podľa zadaných pravdepodobnostných rozdelení.

Každá replikácia simulácie:

1. kuriér začne svoju cestu v meste Žilina v zadanom čase odchodu,
2. postupne navštívi obce podľa zvolenej trasy,
3. medzi jednotlivými mestami systém vyhľadá najrýchlejšiu možnú cestu v cestnej sieti,
4. rýchlosti na jednotlivých úsekoch ciest sa generujú náhodne podľa príslušných pravdepodobnostných rozdelení,
5. ak kuriér prechádza uzlom K po stanovenom čase, jeho rýchlosť sa zníži o náhodne generované percentuálne spomalenie,
6. po návrate späť do Žiliny sa zaznamená celkový čas trvania rozvozu.

Pri hľadaní najkratšej cesty medzi dvoma mestami sa používa algoritmus Dijkstra, ktorý minimalizuje čas prejazdu, nie vzdialenosť. Váhou jednotlivých hrán grafu je teda čas potrebný na prejazd daného úseku, ktorý sa vypočíta zo vzdialenosti a náhodne generovanej rýchlosti.

Výsledky simulácie

Nasledujúca tabuľka zobrazuje výsledné priemerné časy rozvozu pre jednotlivé možné trasy kuriéra.

Trasa	Priemerný čas prechodu (mm:ss)
Z – D – S – R - Z	26:50
Z – D – R – S - Z	**22:16**
Z – S – D – R - Z	24:53
Z – S – R – D - Z	**22:15**
Z – R – S – D - Z	26:49
Z – R – D – S - Z	24:53

Tieto výsledky boli získané pre simulácie, v ktorých kuriér odchádza zo Žiliny o 06:00 a spomalenie dopravy v uzle K začína o 06:30.

Z výsledkov je možné pozorovať, že jednotlivé poradia obsluhy obcí majú výrazný vplyv na celkový čas rozvozu. Najkratší priemerný čas dosahuje trasa $Z - S - R - D - Z$ a veľmi podobný výsledok dosahuje aj trasa $Z - D - R - S - Z$.

Simulácia optimálneho odchodu

Cieľom tejto simulácie je nájsť taký čas odchodu kuriéra zo skladu v Žiline, aby bola pravdepodobnosť dokončenia rozvozu všetkých obcí pred stanoveným časovým limitom čo najbližšia požadovanej hodnote. Keďže čas prejazdu jednotlivých úsekov je náhodný a závisí od generovaných rýchlostí, nie je možné presne určiť vhodný čas odchodu analytickým spôsobom.

Princíp simulácie

Simulácia optimálneho odchodu je založená na opakovanom testovaní rôznych časov odchodu kuriéra. Pre každý kandidátsky čas odchodu sa vykoná Monte Carlo simulácia založená na simulácii obslúženia jednotlivých obcí, ktorá vyhodnotí pravdepodobnosť, že kuriér dokončí rozvoz pred stanoveným časovým limitom.

Každá replikácia simulácie:

1. zvolí sa konkrétny čas odchodu kuriéra zo skladu v Žiline,
2. kuriér začne svoju trasu v meste Žilina,
3. postupne navštívi jednotlivé obce podľa zvolenej trasy a následne sa vráti späť do Žiliny,
4. ak kuriér prechádza uzlom K po čase, kedy začínajú dopravné kolóny, jeho rýchlosť sa zníži o náhodne generované percentuálne spomalenie,
5. po dokončení celej trasy sa vypočíta čas príchodu späť do Žiliny,
6. ak je čas príchodu menší ako stanovený časový limit (*deadline*), replikácia sa považuje za úspešnú.

Po vykonaní všetkých replikácií sa vypočíta pravdepodobnosť úspešného dokončenia rozvozu ako podiel úspešných replikácií k celkovému počtu vykonaných replikácií.

Hľadanie optimálneho času odchodu

Na určenie optimálneho času odchodu sa používa iteratívny postup založený na princípe binárneho vyhľadávania.

Postup algoritmu je nasledovný:

1. určí sa dolná a horná hranica možného času odchodu,
2. vypočíta sa stred intervalu, ktorý predstavuje kandidátsky čas odchodu,
3. pre tento čas sa vykoná Monte Carlo simulácia a vypočíta sa pravdepodobnosť stihnutia rozvozu,
4. ak je vypočítaná pravdepodobnosť menšia ako požadovaná hodnota, znamená to, že kuriér odchádza príliš neskoro a horná hranica intervalu sa posunie skôr,
5. ak je pravdepodobnosť väčšia alebo rovná požadovanej hodnote, algoritmus skúša neskorší čas odchodu posunutím dolnej hranice intervalu,
6. tento proces sa opakuje dovtedy, kým rozdiel medzi dosiahnutou pravdepodobnosťou a požadovanou pravdepodobnosťou nie je menší ako zadaná tolerancia.

Počas simulácie sa zároveň sleduje aj najlepší nájdený čas odchodu, ktorého výsledná pravdepodobnosť je najbližšie k požadovanej hodnote.

Výsledky simulácie

Nasledujúca tabuľka zobrazuje výsledné optimálne časy odchodu pre jednotlivé možné trasy kuriéra.

Trasa	Optimálny čas odchodu	Pravdepodobnosť
Z – D – S – R - Z	07:01:21	80.01%
Z – D – R – S - Z	07:08:37	80.00175%
Z – S – D – R - Z	07:05:33	80.00825%
Z – S – R – D - Z	**07:09:59**	80.00675%
Z – R – S – D - Z	07:02:10	80.007%
Z – R – D – S - Z	07:04:53	80.007%

Tieto výsledky boli získané pre simulácie, kde kuriér sa musí vrátiť o 07:35, spomalenie dopravy v uzle K začína o 06:30 a požadovaná pravdepodobnosť je 80%.

Z výsledkov simulácie vyplýva, že najvhodnejšia trasa z hľadiska optimálneho času odchodu je *Z – S – R – D – Z*, ktorá dosahuje požadovanú pravdepodobnosť stihnutia rozvozu pri najneskoršom možnom čase odchodu. Táto trasa teda umožňuje kuriérovi odísť zo skladu najneskôr a zároveň zachovať požadovanú spoľahlivosť doručenia.

Architektrúra systému

Aplikácia je navrhnutá modulárne a pozostáva z viacerých logických vrstiev, ktoré oddeľujú používateľské rozhranie, simulačné jadro, model systému a pomocné komponenty.

Architektúra systému je rozdelená do niekoľkých hlavných balíkov:

- **gui** – obsahuje triedy používateľského rozhrania,
- **models** – obsahuje implementáciu modelu simulovaného systému,
- **nodes** – reprezentuje štruktúru cestnej siete,
- **probabilities** – implementuje generátory pravdepodobnostných rozdelení,
- **simulations** – obsahuje konkrétne implementácie simulácií,
- **simulations.core** – obsahuje všeobecné jadro Monte Carlo simulácie,
- **simulations.results** – obsahuje triedy reprezentujúce výsledky simulácií.

Simulačné jadro

Základ celej aplikácie tvorí abstraktná trieda *SimulationCore*, ktorá implementuje všeobecné jadro Monte Carlo simulácie. Táto trieda riadi priebeh simulácie, vykonáva jednotlivé replikácie experimentu a zabezpečuje komunikáciu s poslucháčmi priebežných výsledkov.

Simulačné jadro obsahuje základné metódy, ktoré sú volané počas jednotlivých fáz simulácie, napríklad inicializáciu simulácie, vykonanie experimentu, spracovanie výsledkov replikácie a generovanie konečného výsledku.

Konkrétne typy simulácií sú implementované ako potomkovia tejto triedy.

Implementované simulácie

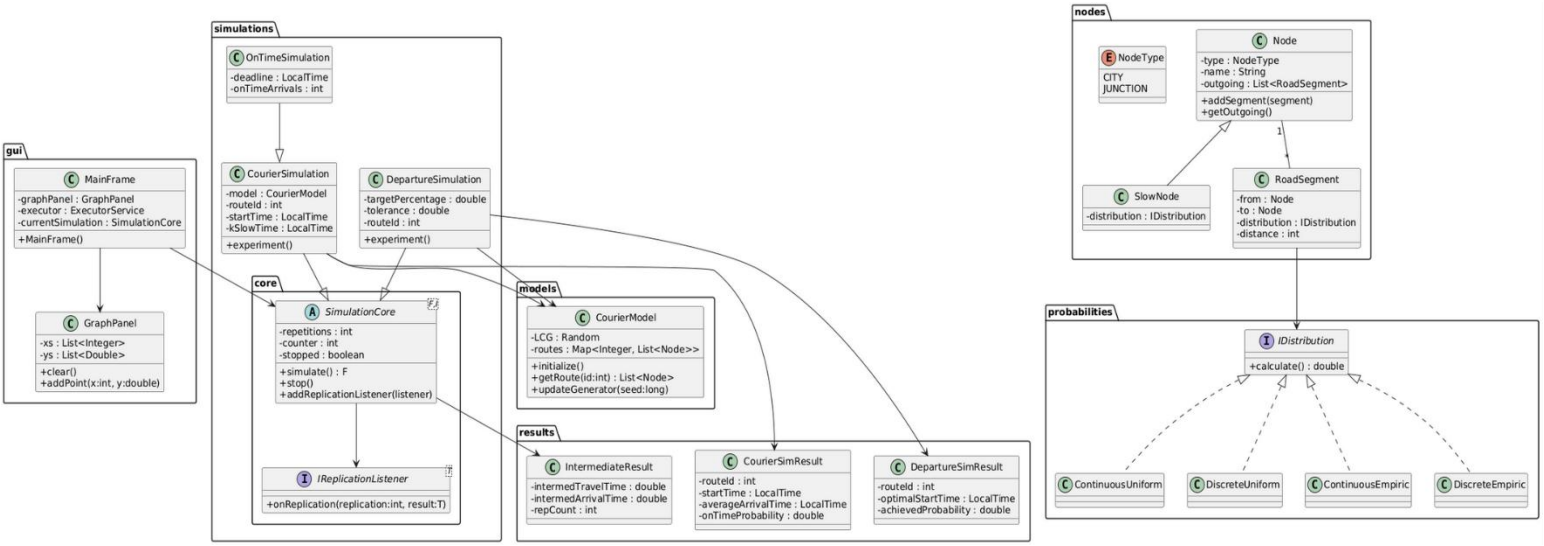
V projekte sú implementované dva hlavné typy simulácií:

- **CourierSimulation** – simulácia obslúženia jednotlivých obcí a výpočet priemerného času návratu kuriéra,
- **DepartureSimulation** – simulácia hľadania optimálneho času odchodu pri požadovanej pravdepodobnosti stihnutia rozvozu.

Trieda *OnTimeSimulation* rozširuje základnú simuláciu a slúži na výpočet pravdepodobnosti, že kuriér stihne dokončiť rozvoz pred stanoveným časovým limitom.

Diagram tried

Nasledujúci diagram tried znázorňuje základnú štruktúru aplikácie a vzťahy medzi jednotlivými triedami. Diagram poskytuje prehľad o architektúre systému a ukazuje rozdelenie tried do jednotlivých balíkov.



4 Diagram tried

Grafické používateľské rozhranie

Používateľské rozhranie je implementované pomocou knižnice Java Swing. Hlavná trieda *MainFrame* zabezpečuje vytvorenie grafického rozhrania, spracovanie vstupných parametrov simulácie a spustenie simulácie. Používateľ môže pomocou rozhrania nastaviť počet replikácií, počiatočný čas odchodu, zvoliť konkrétnu trasu a typ simulácie.

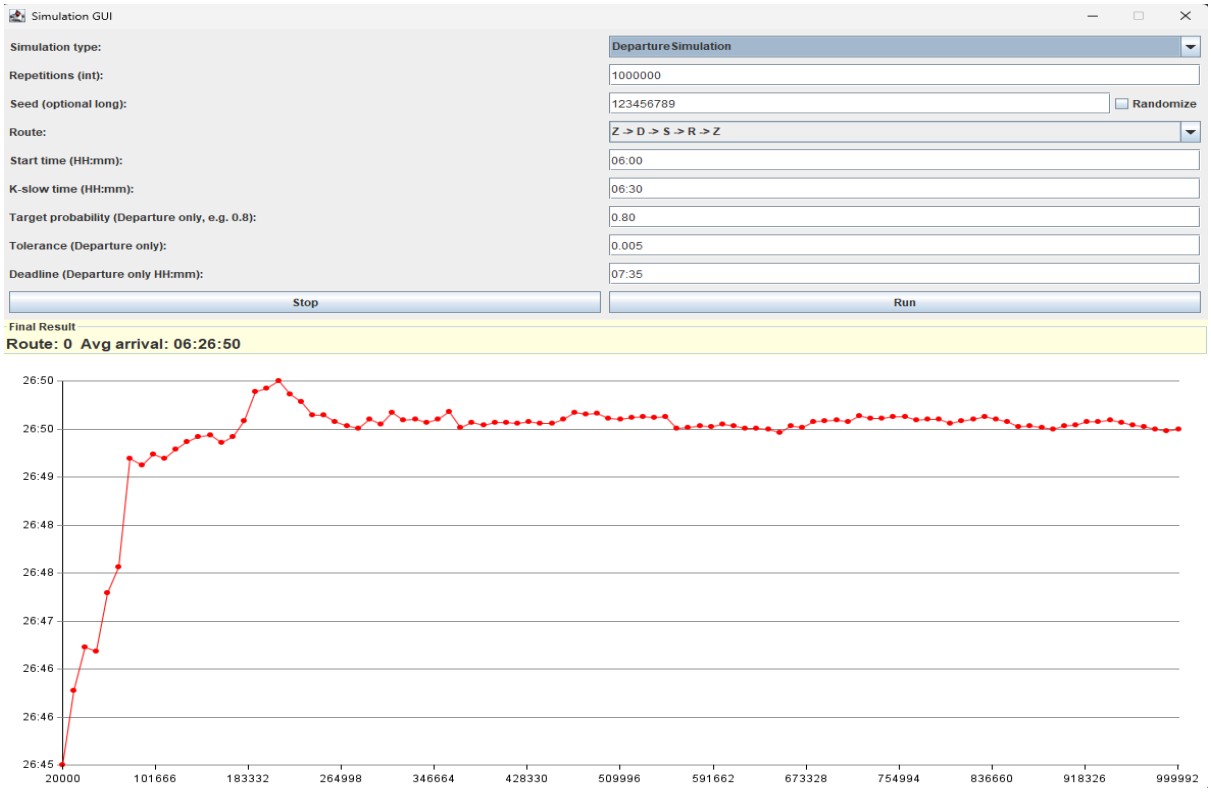
Trieda *GraphPanel* slúži na grafické zobrazenie priebehu simulácie. Počas vykonávania simulácie sa do grafu postupne zapisujú priebežné výsledky, ktoré zobrazujú stabilizáciu priemernej hodnoty sledovaného času.

Program umožňuje nastaviť:

- typ simulácie
- počet replikácií
- generátor násad
- trasu
- čas odchodu
- čas spomalenia
- cieľovú pravdepodobnosť (len pri simulácii optimálneho odchodu)
- toleranciu pravdepodobnosti (len pri simulácii optimálneho odchodu)
- požadovaný čas príchodu (len pri simulácii optimálneho odchodu)

Počas simulácie je možné:

- sledovať graf stabilizácie
- zastaviť simuláciu



5 Grafické používateľské rozhranie

Záver

V práci bol vytvorený simulačný model rozvozu potravín pomocou metódy Monte Carlo. Model umožňuje analyzovať rôzne scenáre dopravy pri náhodných rýchlostiach na jednotlivých úsekoch ciest.

Práca poskytuje dva hlavné simulačné výsledky:

- optimálne poradie obsluhy obcí
- optimálny čas odchodu kuriéra

Vďaka veľkému počtu replikácií je možné získať stabilné odhady priemerného času rozvozu a pravdepodobnosti stihnutia rozvozu. Model je implementovaný modulárne a umožňuje jednoduché rozšírenie o ďalšie scenáre alebo parametre.

Využitie AI

Add a graph to the gui which will be showing the simulation results. There will be only 1 graph with 1 line on it but this graph will be shared by all simulations. But before each simulation run the graph will be cleared so each simulation will start with an empty graph

Planned graph implementation and implemented graph functionality

I'll read MainFrame.java, then add a new `GraphPanel` component and clear/add points before/after simulation.

Reviewed MainFrame.java and modified for simulations

I'll update MainFrame.java: add `GraphPanel`, add it to the center layout, clear it before running, and plot a single point after each simulation.

Modified 2 files

Progress update: I added a simple `GraphPanel` at GraphPanel.java and updated MainFrame to include it, clear it before each run, and plot a result point after simulations.

Next steps: run the app to verify the GUI and graph render correctly, or add labels/unit selection and multiple-point plotting if you want to visualize more data across runs. Want me to run it now?

GPT-5 mini • 1x

```
JSplitPane center = new JSplitPane(JSplitPane.HORIZONTAL_SPLIT);
graphPanel.setPreferredSize(new Dimension(400, 300));
center.setLeftComponent(graphPanel);
center.setRightComponent(scroll);
center.setDividerLocation(420);

add(top, BorderLayout.NORTH);
add(center, BorderLayout.CENTER);

runButton.addActionListener((ActionEvent e) -> {
    resultArea.setText("");
    try {
        int reps = Integer.parseInt(repetitionsField.getText().trim
        int routeId = (Integer) routeBox.getSelectedItem();
        LocalTime startTime = LocalTime.parse(startTimeField.getTe
        LocalTime kSlow = LocalTime.parse(kSlowField.getText().trim

        // clear graph before each run as requested
        graphPanel.clear();

        String selected = (String) simType.getSelectedItem();
        if ("CourierSimulation".equals(selected)) {
            CourierSimResult r = core.runCourierSimulation(reps, r
            resultArea.append("CourierSimulation result:\n");
            resultArea.append("Route id: " + r.routeId() + "\n");
            resultArea.append("Start time: " + r.startTime() + "\n
            resultArea.append("Average travel seconds: " + r.averag
            resultArea.append("On-time probability: " + r.onTimePro

            // plot average travel seconds
            graphPanel.addPoint(r.averageTravelSeconds());
        } else {
```

Rework the entire GUI concept! Drop the use of the GuiCore. Sympli create a gui that will have the followingfeatures. The user can define all the parameter for each one of the 2 simulations. There will be a graph that will display intermediate results of a simulation during it's simulation time so it's progress can be seen visualy. A very small text are above the grap where the final simulation result will be efectivly displayed.

SimulationCore.java:1-54

Searched for files and reviewed simulation implementations

I'll update `MainFrame.java` to remove `GuiCore`, run simulations directly on background thread, register replication listeners to update the graph every 1000 replications, and show final result in a small text area above the graph. I'll apply the patch now.

Edited `MainFrame.java` +78 -38

Done — I removed the `GuiCore` usage and reworked `MainFrame` to:

- Directly build `CourierSimulation` or `DepartureSimulation`.
- Register `IReplicationListener` to receive intermediate results (SimulationCore already fires listeners every 1000 replications).
- Run simulations on a background thread (`ExecutorService`) and update the GUI on the EDT.
- Clear the single shared graph before each run and add intermediate points during the run.
- Display the final result in a small `finalResultArea` above the graph; logs appear on the right.

Files changed:

- `MainFrame.java`

```
if ("CourierSimulation".equals((String) simType.getSelectec
    CourierModel model = new CourierModel();
    model.initialize();
    CourierSimulation sim = new CourierSimulation(reps, moc
    sim.addReplicationListener((int replicationNumber, Inte
        double val = intermediateResult.intermedTravelTime(
        SwingUtilities.invokeLater(() -> {
            graphPanel.addPoint(val);
            logArea.append("rep " + replicationNumber + ",
        });
    });
    executor.submit(() -> {
        CourierSimResult result = sim.simulate();
        SwingUtilities.invokeLater(() -> {
            finalResultArea.setText("Route: " + result.rout

            logArea.append("Simulation finished.\n");
            graphPanel.addPoint((double) result.averageArri
        });
    });
} else {
    double target = Double.parseDouble(targetField.getText(
    double tol = Double.parseDouble(tolField.getText().trim
    LocalTime deadline = LocalTime.parse(deadlineField.getT
    CourierModel model = new CourierModel();
    model.initialize();
    DepartureSimulation sim = new DepartureSimulation(targe
    sim.addReplicationListener((int replicationNumber, Inte
        double val = intermediateResult.intermedArrivalTime
        SwingUtilities.invokeLater(() -> {
            graphPanel.addPoint(val);
            logArea.append("rep " + replicationNumber + ",
        });
    });
    executor.submit(() -> {
        DepartureSimResult result = sim.simulate();
        SwingUtilities.invokeLater(() -> {
            finalResultArea.setText("Route: " + result.rout

            logArea.append("Simulation finished.\n");
            graphPanel.addPoint(result.achievedProbability(
        });
    });
}
```

Add a checkbox next to the seed option in the gui. When checked, the seed will be randomly generated before each simulation run.

Planning implementation steps

I'm about to add a "Randomize seed" checkbox and wire it so when checked a random seed is generated for each run; I'll also add enabling/disabling of the seed input.

Reviewed and updated MainFrame.java

Done — I added a "Randomize" checkbox next to the seed field and wired it so:

- If checked, a new random seed (via `ThreadLocalRandom.current().nextLong()`) is generated for each run.
- The seed text field is disabled while the checkbox is checked.
- Both Courier and Departure simulation branches use the checkbox to decide the seed.

Also left the existing `stop` button behavior as-is (calling `currentSimulation.stop()`). Want me to disable the Run button while a simulation runs and enable Stop only then?

GPT-5 mini • 1x

```java
JTextField seedField = new JTextField("123456789");
JCheckBox randomSeed = new JCheckBox("Randomize");
JPanel seedPanel = new JPanel(new BorderLayout());
seedPanel.add(seedField, BorderLayout.CENTER);
seedPanel.add(randomSeed, BorderLayout.EAST);
top.add(new JLabel("Seed (optional long):"));
top.add(seedPanel);

randomSeed.addActionListener(ae -> seedField.setEnabled(!randomSeed.isSelected()));
```